

Chapter 23 - Resolving collision

In chapter 13 we talked about how we can detect collision. In this chapter we take it a step further. Imagine you have a game where you can move around and there are walls that you can't walk through. To achieve this, we not only need to detect the collision, we also need to make sure the player stops moving and does not get stuck inside the wall. We need to push the player back.

Before we start, I want to tell you that resolving collision is very hard, and it's something that even professionals have trouble dealing with. Look at speedruns for example. There are a lot of glitches where you are able to clip through walls and such. The collision we're going to make is pretty solid but far from perfect. And don't feel bad if you have a hard time understanding what we're doing here.

For this tutorial we need two things: A player that can walk in 4 directions, and some walls. Let's create a base class for both of these. An Entity class.

So we have the following files:

- main.lua
- player.lua
- wall.lua
- entity.lua
- classic.lua

Let's start with the Entity class. It has an x, y, width and height. Let's use an image for our walls and player, and use the width and height of that image.

--! file: entity.lua

Entity = Object:extend()

function Entity:new(x, y, image_path)

self.x = x

self.y = y

self.image = love.graphics.newImage(image_path)

self.width = self.image:getWidth()

```
    self.height = self.image:getHeight()
end
```

```
function Entity:update(dt)
    -- We'll leave this empty for now.
end
```

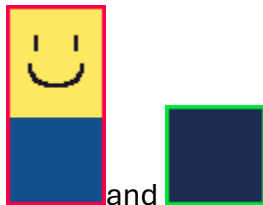
```
function Entity:draw()
    love.graphics.draw(self.image, self.x, self.y)
end
```

And let's add collision detection. Do you still know how to do it and why it works like that? Perhaps give Chapter 13 another read.

```
function Entity:checkCollision(e)
    -- e will be the other entity with which we check if there is collision.
    -- This is the final compact version from chapter 13

    return self.x + self.width > e.x
    and self.x < e.x + e.width
    and self.y + self.height > e.y
    and self.y < e.y + e.height
end
```

Now that we have our subclass, we can create our player and wall. Here are the images that we will be using (the colorful borders help us see if there is collision):



```
--! file: player.lua
```

```
Player = Entity:extend()
```

```
function Player:new(x, y)
```

```
    Player.super.new(self, x, y, "player.png")
```

```
end
```

```
--! file: wall.lua
```

```
Wall = Entity:extend()
```

```
function Wall:new(x, y)
```

```
    Wall.super.new(self, x, y, "wall.png")
```

```
end
```

Let's make it so that we can move our player using the arrow keys.

```
--! file: player.lua
```

```
Player = Entity:extend()
```

```
function Player:new(x, y)
```

```
    Player.super.new(self, x, y, "player.png")
```

```
end
```

```
function Player:update(dt)
```

```
    if love.keyboard.isDown("left") then
```

```
        self.x = self.x - 200 * dt
```

```
    elseif love.keyboard.isDown("right") then
```

```
        self.x = self.x + 200 * dt
```

```
    end
```

```
if love.keyboard.isDown("up") then
    self.y = self.y - 200 * dt
elseif love.keyboard.isDown("down") then
    self.y = self.y + 200 * dt
end
end
```

Okay now let's create these objects in main.lua

--! file: main.lua

```
function love.load()
    -- First require classic, since we use it to create our classes.
    Object = require "classic"

    -- Second require Entity, since it's the base class for our other classes.
    require "entity"
    require "player"
    require "wall"

    player = Player(100, 100)
    wall = Wall(200, 100)
end

function love.update(dt)
    player:update(dt)
    wall:update(dt)
end

function love.draw()
```

```
player:draw()
```

```
wall:draw()
```

```
end
```

Now we can walk around, and of course we can walk through our wall. So here's an idea, what if we keep track of our previous position, and then whenever we hit a wall, we just spawn back to that position. That way the player can never go through a wall.

We need to make some changes to our Entity class. We add a last object for our previous positions, and we add a function that checks if there is collision and if so puts us back to our previous position.

Since there are a lot of small changes to the code in this chapter, I added ---- ADD THIS to places where you need to add/change something to the code.

```
--! file: entity.lua
```

```
Entity = Object:extend()
```

```
function Entity:new(x, y, image_path)
```

```
    self.x = x
```

```
    self.y = y
```

```
    self.image = love.graphics.newImage(image_path)
```

```
    self.width = self.image:getWidth()
```

```
    self.height = self.image:getHeight()
```

```
---- ADD THIS
```

```
    self.last = {}
```

```
    self.last.x = self.x
```

```
    self.last.y = self.y
```

```
-----
```

```
end
```

-- I have a habit of passing dt regardless of if we need it.

---- ADD THIS

function Entity:update(dt)

-- Set the current position to be the previous position

self.last.x = self.x

self.last.y = self.y

end

function Entity:resolveCollision(e)

if self:checkCollision(e) then

-- Reset the position.

self.x = self.last.x

self.y = self.last.y

end

end

We need to call the base class function in our Player class to make this work.

--! file: player.lua

function Player:update(dt)

-- It's important that we do this before changing the position

---- ADD THIS

Player.super.update(self, dt)

if love.keyboard.isDown("left") then

```

    self.x = self.x - 200 * dt

elseif love.keyboard.isDown("right") then

    self.x = self.x + 200 * dt

end


if love.keyboard.isDown("up") then

    self.y = self.y - 200 * dt

elseif love.keyboard.isDown("down") then

    self.y = self.y + 200 * dt

end

end

```

And we need to call the resolveCollision() function in main.lua.

--! file: main.lua

```

function love.update(dt)

    player:update(dt)

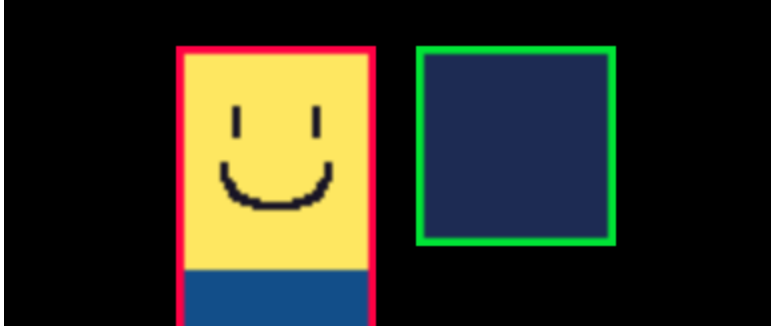
    wall:update(dt)

    player:resolveCollision(wall)

end

```

Try it out. You'll notice that it works... sort of. If you look closely you might see that there is sometimes a small gap between the player and the wall. This is because the player moves so fast that it skips over that part, touches the wall and gets sent back to its previous position.



Instead of placing the player on its previous position, we should move the player until it doesn't touch the wall anymore. So basically we push it away. Not back all the way to its previous position but just enough so that it doesn't overlap with the wall anymore (so that the edges are touching).

For this we need to know two things:

- How far should we move the player so that it doesn't overlap with the wall anymore
- In which direction should we push the player away from the wall?

For now let's assume that the player is coming from the left, and then later we can make it so that it works for all directions.

So we need to calculate how much the player overlaps with the wall. We can do this by calculating the right side of the player - the left side of the wall. If the player's right side is on x-position 225 and the wall's left side is on x-position 205, then the player should be pushed to the left with 20 pixels.

--! file: entity.lua

function Entity:resolveCollision(e)

if self:checkCollision(e) **then**

-- The player's right side is x + width

-- The wall's left side is x

-- Calculate the difference and subtract that from the player's position

local pushback = self.x + self.width - e.x

self.x = self.x - pushback

end

end

Now there's no gap anymore. And since we only push the player away horizontally it can now move vertically while touching the wall.

Let's make it work for all directions. To figure out from which direction the player is coming from we can use our previous position. Because unless we came from the corner, our previous position was already vertically or horizontally aligned with the wall.



If we were to come from the left, we are already vertically aligned. We are already colliding with the wall on a vertical level. Remember the four conditions from chapter 13? Right before we touch the wall from the left, two of those conditions are met.

A's bottom side is further to the bottom than B's top side.

A's top side is further to the top than B's bottom side.

In this case, we must move horizontally to collide with the wall, meaning we know that the player came from either the left or right side.

So we check if on our previous position if we were already colliding horizontally or vertically with the wall and based on that we can determine whether we should push the player away vertically or horizontally.

function Entity:wasVerticallyAligned(e)

-- It's basically the collisionCheck function, but with the x and width part removed.

-- It uses last.y because we want to know this from the previous position

return self.last.y < e.last.y + e.height **and** self.last.y + self.height > e.last.y

end

```
function Entity:wasHorizontallyAligned(e)
```

```
-- It's basically the collisionCheck function, but with the y and height part removed.
```

```
-- It uses last.x because we want to know this from the previous position
```

```
return self.last.x < e.last.x + e.width and self.last.x + self.width > e.last.x
```

```
end
```

```
function Entity:resolveCollision(e)
```

```
if self:checkCollision(e) then
```

```
    if self:wasVerticallyAligned(e) then
```

```
        elseif self:wasHorizontallyAligned(e) then
```

```
            end
```

```
        end
```

```
end
```

Now that we know this, we need to check from which side they're touching. A simple way is to check the center of both the wall and the player. In the case it was already vertically aligned: If the center of the player is more to the left than the center of the wall, we push it to the left. Let's try it out.

```
function Entity:resolveCollision(e)
```

```
if self:checkCollision(e) then
```

```
    if self:wasVerticallyAligned(e) then
```

```
        if self.x + self.width/2 < e.x + e.width/2 then
```

```
            -- pusback = the right side of the player - the left side of the wall
```

```
            local pushback = self.x + self.width - e.x
```

```
            self.x = self.x - pushback
```

```

else

    -- pusback = the right side of the wall - the left side of the player

    local pushback = e.x + e.width - self.x

    self.x = self.x + pushback

end

elseif self:wasHorizontallyAligned(e) then

    if self.y + self.height/2 < e.y + e.height/2 then

        -- pusback = the bottom side of the player - the top side of the wall

        local pushback = self.y + self.height - e.y

        self.y = self.y - pushback

    else

        -- pusback = the bottom side of the wall - the top side of the player

        local pushback = e.y + e.height - self.y

        self.y = self.y + pushback

    end

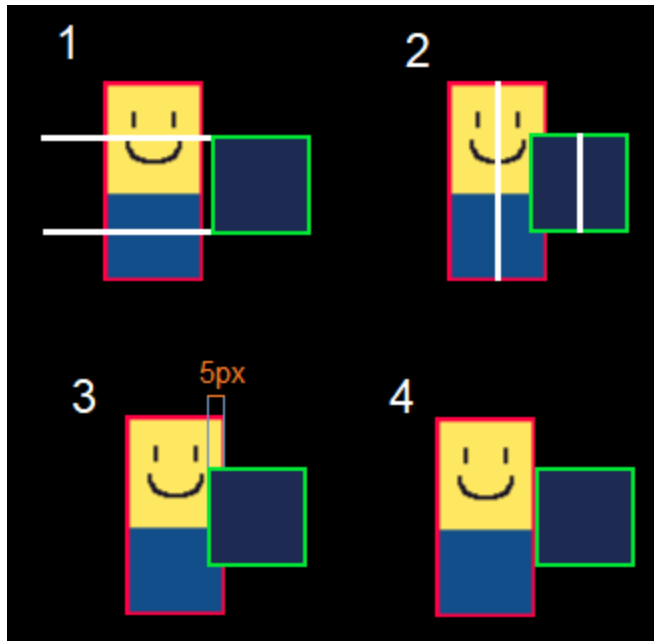
end

end

end

```

It works! But okay that's a lot of information at once. Let's recap that.



1. The player and the wall are already on the same height. With this information we can determine that the player must've moved horizontally to collide with the wall.
2. The center of the player is more to the left than the wall's center, so we should push it to the left.
3. The player's right side is 5 pixels to the right of the wall's left side. Or in more simple words: The player needs to be pushed back 5 pixels so that he doesn't overlap with the wall anymore.
4. The player is pushed back correctly! I'm at a loss for words.. we did it!

So that works. But there is one more thing I want to fix. Because right now we call `player:resolveCollision(wall)` but if we were to call it the other way around `wall:resolveCollision(player)` it wouldn't. Of course in this instance we know that the wall is stronger than the player, but let's assume we don't know that.

To fix this we can add a variable called `strength`, and the property with lower strength gets pushed away.

```
Entity = Object:extend()
```

```
function Entity:new(x, y, image_path)
```

```
    self.x = x
```

```

self.y = y

self.image = love.graphics.newImage(image_path)

self.width = self.image:getWidth()

self.height = self.image:getHeight()


self.last = {}

self.last.x = self.x

self.last.y = self.y


-- Add a default value to the variable
---- ADD THIS

self.strength = 0

-----

end

--! file: wall.lua

Wall = Entity:extend()

```

```

function Wall:new(x, y)

    Wall.super.new(self, x, y, "wall.png")

    -- Give the wall a strength of 100

    self.strength = 100

end

```

Now we check the value in `Entity:resolveCollision(e)` and if the value of `self.strength` is higher than the value of `e.strength`, then we call `e:resolveCollision(self)`. So we turn the roles around.

```

--! file: entity.lua

function Entity:resolveCollision(e)

```

---- ADD THIS

```
if self.strength > e.strength then  
    e:resolveCollision(self)  
    -- Return because we don't want to continue this function.  
  
    return  
  
end
```

```
if self:checkCollision(e) then  
    if self:wasVerticallyAligned(e) then  
        if self.x + self.width/2 < e.x + e.width/2 then  
            local pushback = self.x + self.width - e.x  
            self.x = self.x - pushback  
  
        else  
            local pushback = e.x + e.width - self.x  
            self.x = self.x + pushback  
  
        end  
  
    elseif self:wasHorizontallyAligned(e) then  
        if self.y + self.height/2 < e.y + e.height/2 then  
            local pushback = self.y + self.height - e.y  
            self.y = self.y - pushback  
  
        else  
            local pushback = e.y + e.height - self.y  
            self.y = self.y + pushback  
  
        end  
  
    end  
  
end  
  
end
```

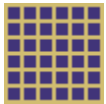
end

And now it doesn't matter in which order we call the function.

Pushing a box

The wall pushes the player away. But what if we want the player to move something? To achieve this we need to make some changes to our code.

First let's create a Box class. We can use this image for it:



```
--! file: box.lua
```

```
Box = Entity:extend()
```

```
function Box:new(x, y)
```

```
    Box.super.new(self, x, y, "box.png")
```

```
end
```

Next we want to add the box, and while we're at it, let's make an objects table.

```
--! file: main.lua
```

```
function love.load()
```

```
    Object = require "classic"
```

```
    require "entity"
```

```
    require "player"
```

```
    require "wall"
```

```
    -- Require box!
```

```
    require "box"
```

```
    player = Player(100, 100)
```

```
wall = Wall(200, 100)
```

```
box = Box(400, 150)
```

```
objects = {}
```

```
table.insert(objects, player)
```

```
table.insert(objects, wall)
```

```
table.insert(objects, box)
```

end

Now we can use an ipairs loop for updating and drawing the objects, but what about resolving the collision? Because we want all the objects to check collision with all other objects. How can we achieve this? We could simply have two for-loops, where we also need to check if it's not at the same object. Like so:

-- Example, no need to copy

```
for i,v in ipairs(objects) do
```

```
    for j,w in ipairs(objects) do
```

```
        -- Make sure they're not the same
```

```
        -- Else it would be resolving collision with itself
```

```
        if v ~= w then
```

```
            v:resolveCollision(w)
```

```
        end
```

```
    end
```

```
end
```

The problem with this however is that we're now checking collision twice for each object. After we've called `player:resolveCollision(wall)` we don't need to call `wall:resolveCollision(player)` but that's basically what we're doing. So to prevent this we go through the list of objects in the first loop, and in the second loop we go through the list, starting with the object next to the object in the first loop.

```
function love.update(dt)
```

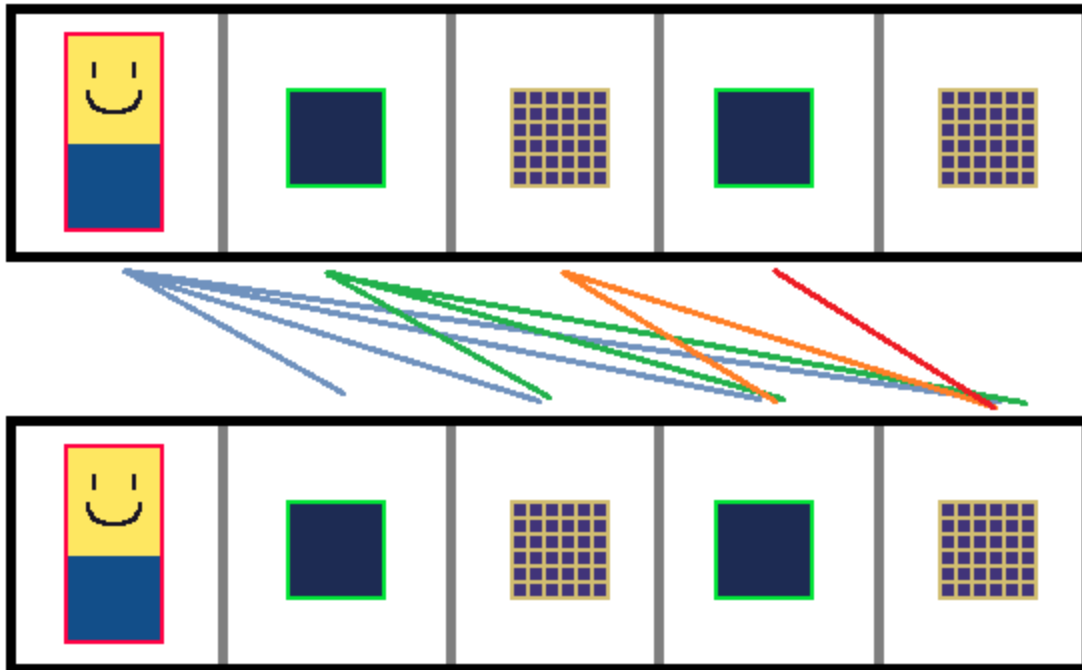
```

-- Update all the objects
for i,v in ipairs(objects) do
    v:update(dt)
end

-- Go through all the objects (except the last)
for i=1,#objects-1 do
    -- Go through all the objects starting from the position i + 1
    for j=i+1,#objects do
        objects[i]:resolveCollision(objects[j])
    end
end
end

function love.draw()
    -- Draw all the objects
    for i,v in ipairs(objects) do
        v:draw()
    end
end

```



(I added an extra wall and box to make it more clear)

So the first object, the player, resolves collision with the next four objects.

The second object, the wall, doesn't need to resolve collision with the player, because this has already been done previously by the player.

Etc.

Anyway, we can't push the box yet. We need to make the player stronger than the box.

--! file: *player.lua*

Player = Entity:extend()

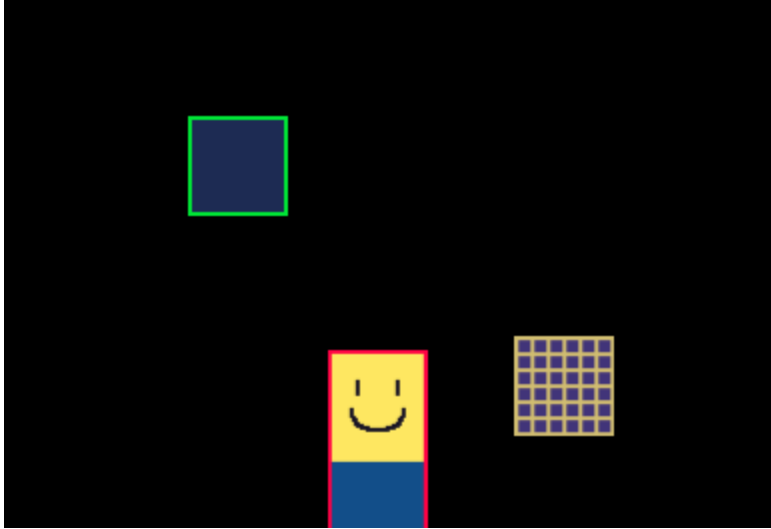
function Player:new(x, y)

 Player.super.new(self, x, y, "player.png")

 self.strength = 10

end

And now we're able to push the box. Awesome, so are we done? Nope! Try to push the box against the wall. That doesn't look right, does it?



Why does this happen? Because the player pushes the box against the wall, but the wall pushes the box back to player. What you want to happen is that both the box and the player stop moving as it collides with the wall. To fix this we need to pass the strength of the wall onto the box, so that when the box is strong enough to push the player away.

But this strength should only be temporary, because else you wouldn't be able to push the box as soon as you touch it, as it gets the same strength as the player. So we create separate property for this called tempStrength.

```
--! file: entity.lua
```

```
Entity = Object:extend()
```

```
function Entity:new(x, y, image_path)
```

```
    self.x = x
```

```
    self.y = y
```

```
    self.image = love.graphics.newImage(image_path)
```

```
    self.width = self.image:getWidth()
```

```
    self.height = self.image:getHeight()
```

```
    self.last = {}
```

```
    self.last.x = self.x
```

```
self.last.y = self.y
```

```
-- Add a default value to the variable
```

```
self.strength = 0
```

```
-- Temp is short for temporary
```

```
---- ADD THIS
```

```
self.tempStrength = 0
```

```
-----
```

```
end
```

```
function Entity:update(dt)
```

```
self.last.x = self.x
```

```
self.last.y = self.y
```

```
-- Reset the strength
```

```
---- ADD THIS
```

```
self.tempStrength = self.strength
```

```
-----
```

```
end
```

```
function Entity:resolveCollision(e)
```

```
-- Compare the tempStrength
```

```
---- ADD THIS
```

```
if self.tempStrength > e.tempStrength then
```

```
    e:resolveCollision(self)
```

```
    -- Return because we don't want to continue this function.
```

```
return
```

end

if self:checkCollision(e) **then**

-- Copy the tempStrength

---- ADD THIS

 self.tempStrength = e.tempStrength

if self:wasVerticallyAligned(e) **then**

if self.x + self.width/2 < e.x + e.width/2 **then**

local pushback = self.x + self.width - e.x

 self.x = self.x - pushback

else

local pushback = e.x + e.width - self.x

 self.x = self.x + pushback

end

elseif self:wasHorizontallyAligned(e) **then**

if self.y + self.height/2 < e.y + e.height/2 **then**

local pushback = self.y + self.height - e.y

 self.y = self.y - pushback

else

local pushback = e.y + e.height - self.y

 self.y = self.y + pushback

end

end

end

end

Almost done! The problem now is that the player pushes box into the wall, and the wall pushes the box back into the player, but after that we never resolve collision between the box and the player again. Because we already did so in that loop. So what we should do is keep checking collision between the objects until all collision is resolved.

Let's make it so that the function Entity:resolveCollision(e) returns true or false based on if there was collision. If there is collision, we go through the objects again and check one more time if there is collision.

function Entity:resolveCollision(e)

-- Compare the tempStrength

if self.tempStrength > e.tempStrength **then**

-- We need to return the value that this method returns

-- Else it will never reach main.lua

---- ADD THIS

return e:resolveCollision(self)

end

if self:checkCollision(e) **then**

self.tempStrength = e.tempStrength

if self:wasVerticallyAligned(e) **then**

if self.x + self.width/2 < e.x + e.width/2 **then**

local pushback = self.x + self.width - e.x

self.x = self.x - pushback

else

local pushback = e.x + e.width - self.x

self.x = self.x + pushback

```

    end

elseif self:wasHorizontallyAligned(e) then
    if self.y + self.height/2 < e.y + e.height/2 then
        local pushback = self.y + self.height - e.y
        self.y = self.y - pushback
    else
        local pushback = e.y + e.height - self.y
        self.y = self.y + pushback
    end
end

end

-- There was collision! After we've resolved the collision return true
---- ADD THIS

return true

-----

end

-- There was NO collision, return false
-- (Though not returning anything would've been fine as well)
-- (Since returning nothing would result in the returned value being nil)
---- ADD THIS

return false

-----

end

```

Now we want to make it so that it keeps checking collision as long as there has been collision resolved. For this we can use a *while-loop*. This keeps looping as long as the statement is true. Careful though! If you use a while-loop the wrong way, it can create an endless loop and it will crash your game. Because of this, let's also add a limit to the number of loops. It just might happen that in a weird occasion we keep getting collision

somehow and we get stuck in an infinite loop. It's better to be safe. If after a 100 loops there is still collision we break the while-loop.

```
--! file:main.lua
```

```
function love.update(dt)
```

```
    -- Update all the objects
```

```
    for i,v in ipairs(objects) do
```

```
        v:update(dt)
```

```
    end
```

```
    local loop = true
```

```
    local limit = 0
```

```
    while loop do
```

```
        -- Set loop to false, if no collision happened it will stay false
```

```
        loop = false
```

```
        limit = limit + 1
```

```
        if limit > 100 then
```

```
            -- Still not done at loop 100
```

```
            -- Break it because we're probably stuck in an endless loop.
```

```
            break
```

```
        end
```

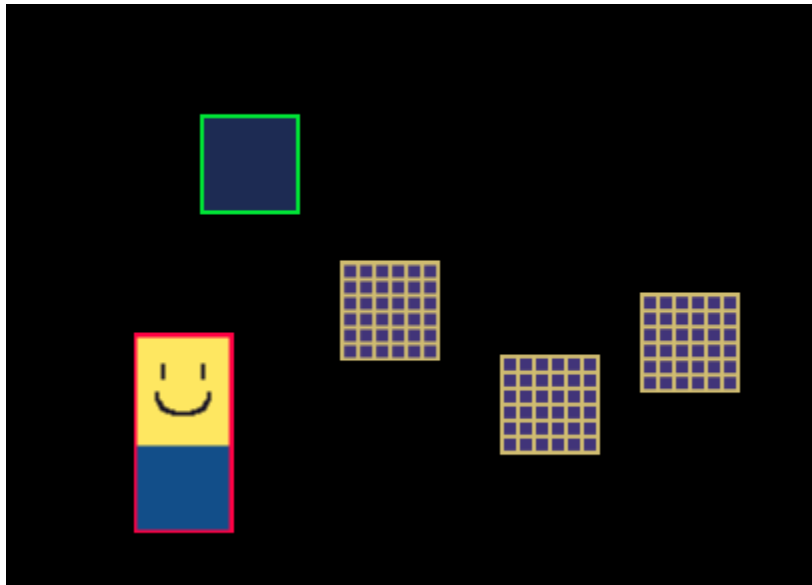
```
    for i=1,#objects-1 do
```

```
        for j=i+1,#objects do
```

```
            local collision = objects[i]:resolveCollision(objects[j])
```

```
        if collision then
            loop = true
        end
    end
end
end
end
end
```

It works! It finally works! We can even have multiple boxes and it still works!



Libraries

Our collision works great, but far from perfect. For more advanced collision handling check out **bump**. For collision with shapes other than rectangles check out **HC**.

- [Bump](#)
- [HC](#)

Summary

We can resolve collision by pushing the player out of the object it's colliding with. We can use a strength property to determine which object should push which. We can pass on this strength so that when you push a box into a wall, the box pushed back by the wall will push back the player. We should keep resolving collisions with the objects until there are no more collision happening. With a while-loop we can loop through something as long as the statement used is true. We should be careful with using a while-loop, as an endless loop will make our game crash.